

```
# import required packages
import torch                                # core PyTorch library for tensors and autograd
import torchvision                          # datasets and image transforms
from torch.utils.data import DataLoader      # utility for iterating over datasets in batches
from matplotlib import pyplot              # used for plotting images, curves and histograms
from sklearn.metrics import ConfusionMatrixDisplay # used later for the confusion matrix plot

# Device configuration: pick GPU if one is available, otherwise fall back to CPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
if torch.cuda.is_available():
    print('Using GPU')
else:
    print('Using CPU. Warning: This will be slow!')
```

Using GPU

Task 2.1: Datasets and Data Loaders

We will make use of `torchvision.datasets.EMNIST` dataset and `torchvision.datasets.FashionMNIST` dataset. EMNIST has two splits, `mnist` and `letters`, with 10 labels of digit images and 26 labels of letter images respectively. FashionMNIST has 10 labels of merchandise images. However, for the training of the Auto-Encoder (AE) we do not make use of the labels of the dataset, but we only utilize the images. For testing purposes, we assign label `-1` to all unknown samples -- note that this is different from the theoretical part in the lecture, where we assign label `0` as unknown.

These datasets can be split into train sets and test sets by the default implementation. We will use solely the MNIST split for training and validation of the AE network. To evaluate this network for out-of-distribution detection, we will use the test sets from EMNIST letters and FashionMNIST respectively.

1. Create the `train_dataset` for the EMNIST `mnist` split.
2. Create test datasets for all datasets. For the test data loaders set `shuffle=True` for the sampling of images to be simpler, even though normally we would not shuffle the test set.
3. Create dataloaders for all datasets.

```
# Hyperparameters
batch_size = 128

# transform for FashionMNIST dataset: converts PIL image to a float tensor in [0,1]
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
])

# transform for EMNIST dataset
def rotate_image(image):
    """Transpose the last two axes of a tensor image to correct EMNIST orientation."""
    # Correct orientation of images
    return image.transpose(2, 1)
emnist_transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    rotate_image,
])

# training set of mnist digits
train_dataset = torchvision.datasets.EMNIST(root='./data', split='mnist', train=True, download=True, transform=emnist_transform)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

# test dataset of mnist digits
test_dataset = torchvision.datasets.EMNIST(root='./data', split='mnist', train=False, download=True, transform=emnist_transform)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)

# Create dataset and dataloader for EMNIST letters
emnist_letters_dataset = torchvision.datasets.EMNIST(root='./data', split='letters', train=False, download=True, transform=emnist_transform)
# Assign label -1 to all EMNIST letter samples
emnist_letters_dataset.targets = torch.full_like(emnist_letters_dataset.targets, -1)

emnist_letters_loader = DataLoader(emnist_letters_dataset, batch_size=batch_size, shuffle=True)

# Create dataset and dataloader for FashionMNIST
fashion_mnist_dataset = torchvision.datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)
# Assign -1 label to all FashionMNIST samples
fashion_mnist_dataset.targets = torch.full_like(fashion_mnist_dataset.targets, -1)
fashion_mnist_loader = DataLoader(fashion_mnist_dataset, batch_size=batch_size, shuffle=True)

# print the number of samples in each dataset for a sanity check
print(f"Number of training samples: {len(train_dataset)}")
```

```
print(f"Number of test samples: {len(test_dataset)}")
print(f"Number of EMNIST letters samples: {len(emnist_letters_dataset)}")
print(f"Number of FashionMNIST samples: {len(fashion_mnist_dataset)}")
```

```
Number of training samples: 60000
Number of test samples: 10000
Number of EMNIST letters samples: 20800
Number of FashionMNIST samples: 10000
```

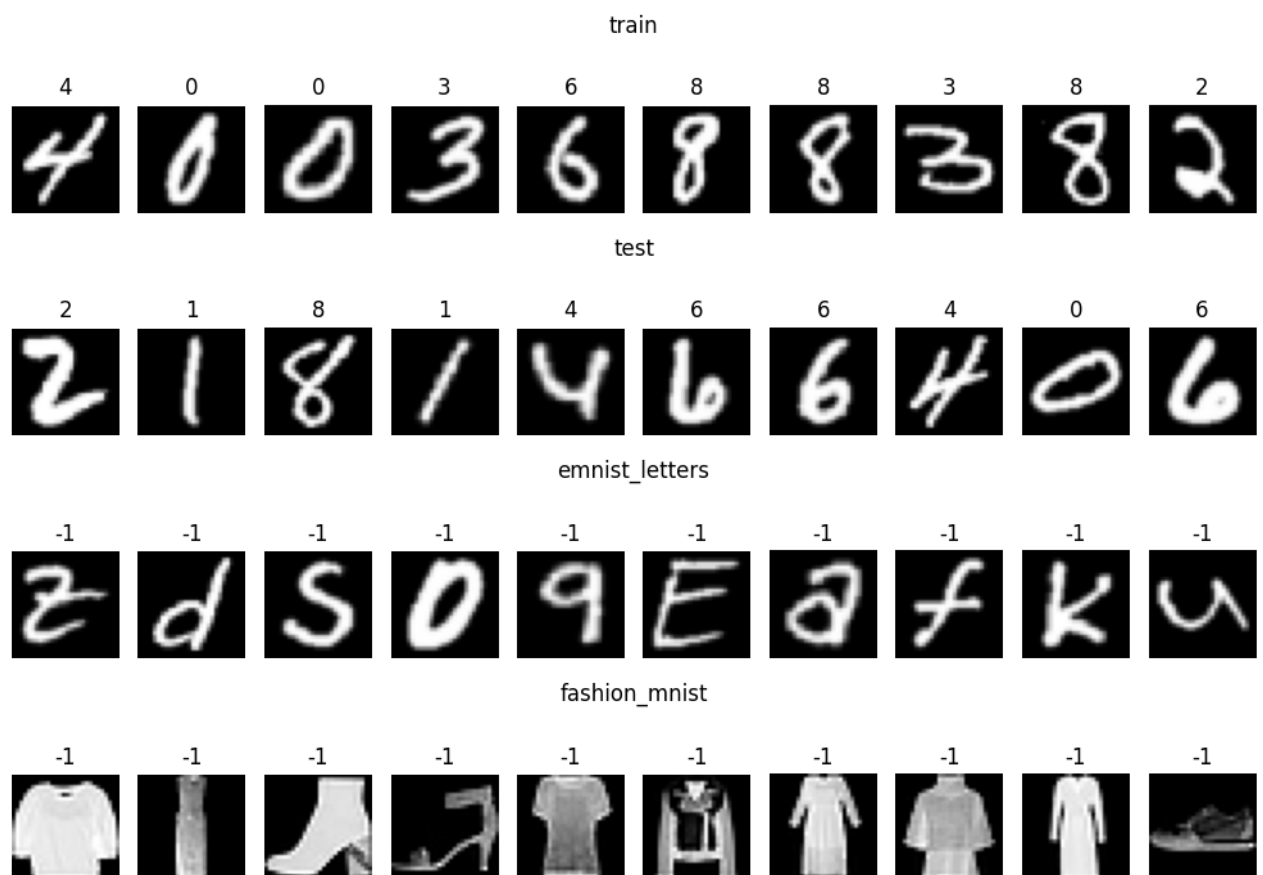
We look at some images to get an idea of the data we are working with. Notably, the letters are much more similar to the training data than the FashionMNIST data, therefore we call the letters near-OOD while the FashionMNIST data is far-OOD.

Note: We only look at the test sets now, to make sure that they are implemented correctly.

```
# look at some images from each dataset
def show_images(data_loader, title, nr_images = 10):
    """Plot a row of sample images from the first batch of a data loader."""
    # sample images from the dataset by grabbing a single batch
    images, labels = next(iter(data_loader))
    # create a figure with one subplot per image
    fig, axs = pyplot.subplots(1, nr_images, figsize=(nr_images, 2))
    fig.suptitle(title)
    # iterate and display each image with its label as the subplot title
    for i in range(nr_images):
        axs[i].imshow(images[i][0], cmap='gray')
        axs[i].set_title(labels[i].item())
        axs[i].axis('off')
    fig.tight_layout()
    pyplot.show()

# mapping from a human-readable name to the corresponding loader
datasets = {
    'train': train_loader,
    'test': test_loader,
    'emnist_letters': emnist_letters_loader,
    'fashion_mnist': fashion_mnist_loader
}

# visualize a few samples from every dataset
for name, loader in datasets.items():
    show_images(loader, name)
```



The auto-encoder network is composed of two parts: the encoder that transforms the input image to a deep feature representation; and the decoder that produces an image from such a deep feature.

For the encoder $\mathcal{E}$, we will use a convolutional network and perform down-sampling via striding. After each convolution, we apply the ReLU activation. The output of the encoder is a $K = 10$ dimensional deep feature representation. The complete encoder network topology can be found below in Topology 1(a).

The decoder $\mathcal{D}$ performs the inverse operations of the encoder. A fully-connected layer is used to increase the number of samples to the same size as the output of the flattening of the encoder. Then, the flattening needs to be undone by reshaping the vector into the correct dimensionality, followed by a ReLU activation. A fractionally-strided convolutional layer increases the intermediate representation by a factor of 2. Note that the fractionally-strided convolution is implemented in `torch.nn.ConvTranspose2d`, and the `stride` parameter should have the same value as for the encoder. Additionally, the `torch.nn.ConvTranspose2d` has a parameter `output_padding` which needs to be adapted to reach the correct output shape (see Test 2). After this layer, we perform another ReLU activation and another fractionally-strided convolution to arrive at the original input dimension. The complete decoder network topology can be found below in Topology 1(b).

Finally, we combine the two sub-networks into one auto-encoder network. While there exist several possibilities for doing this, we will implement a third `torch.nn.Module` that contains an instance of the encoder and an instance of the decoder.

Topology 1: Network configurations of the (a) encoder and (b) decoder networks

(a) Encoder Network

- 2D convolutional layer with Q_1 channels, kernel size 5×5 , **stride 2** and padding 2
- activation function ReLU
- 2D convolutional layer with Q_2 channels, kernel size 5×5 , **stride 2** and padding 2
- flatten layer to convert the convolution output into a vector
- activation function ReLU
- fully-connected layer with the correct number of inputs and K outputs

(b) Decoder Network

- fully-connected layer with K inputs and the correct number of outputs
- activation function ReLU
- reshaping to convert the vector into a convolution input
- 2D **fractionally-strided convolutional** layer with Q_2 channels, kernel size 5×5 , stride 2 and padding 2
- activation function ReLU
- 2D **fractionally-strided convolutional** layer with Q_1 channels, kernel size 5×5 , stride 2 and padding 2

Task 2.2: Encoder Network

Implement the encoder network for given parameters Q_1 , Q_2 , and K as given in Topology 1(a). Implement a network class that derives from `torch.nn.Module` and implement the `__init__` and the `forward` methods.

```
import torch.nn as nn

class Encoder(torch.nn.Module):
    """Encoder sub-network of the convolutional auto-encoder.

    Constructor args:
        Q1, Q2: the number of convolutional channels for layer 1 and 2
        K: The dimensionality of the embedding vector
    """
    def __init__(self, Q1, Q2, K):
        # call base class constructor
        super(Encoder, self).__init__()
        # convolutional define layers
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=Q1, kernel_size=5, stride=2, padding=2)
        self.conv2 = nn.Conv2d(in_channels=Q1, out_channels=Q2, kernel_size=5, stride=2, padding=2)
        # activation functions will be re-used for the different stages
        self.act = nn.ReLU()
        # define fully-connected layers
        self.flatten = nn.Flatten()
        self.fc = nn.Linear(in_features=Q2*7*7, out_features=K)

    def forward(self, x):
        """Run a forward pass producing the deep feature representation.

        Args:
            x: An input batch of size Bx1xHxW where B is the batch size and H=W=28 is the input image dimension

        Returns:
            The embedding batch of dimension BxK
        """
        # get the deep feature representation
```

```

x = self.act(self.conv1(x))
x = self.act(self.conv2(x))
x = self.flatten(x)
deep_feature = self.fc(x)
return deep_feature

```

Task 2.3: Decoder Network

Implement the decoder network for given parameters Q_1 , Q_2 , and K as given in Topology 1(b). Implement a network class that derives from `torch.nn.Module` and implement the `__init__` and the `forward` methods. The output of the decoder network is supposed to have values in the range $[0, 1]$, similar to the input values. We need to make sure that only these values can be achieved. Think of possible ways of doing that, and apply the way that seems most reasonable.

```

class Decoder(torch.nn.Module):
    """Decoder sub-network of the convolutional auto-encoder.

    Constructor args:
        Q1, Q2: the number of convolutional channels for layer 1 and 2 of the encoder network
        K: The dimensionality of the embedding vector
    """
    def __init__(self, Q1, Q2, K):
        # call base class constructor
        super(Decoder, self).__init__()
        # fully-connected layer
        self.fc = nn.Linear(in_features=K, out_features=Q2*7*7)
        # convolutional layers
        self.deconv1 = nn.ConvTranspose2d(in_channels=Q2, out_channels=Q1, kernel_size=5, stride=2, padding=2, output_paddi
        self.deconv2 = nn.ConvTranspose2d(in_channels=Q1, out_channels=1, kernel_size=5, stride=2, padding=2, output_paddi
        # activation function (for intermediate layers)
        self.act = nn.ReLU()
        # Final activation to ensure output is in [0,1]
        self.final_act = nn.Sigmoid()

    def forward(self, x):
        """Run a forward pass producing a reconstructed image.

        Args:
            x: The embedding of dimension BxK

        Returns:
            A batch of reconstructed images of size Bx1xHxW
        """
        # reconstruct the output image
        x = self.act(self.fc(x))
        x = x.view(-1, self.deconv1.in_channels, 7, 7)
        x = self.act(self.deconv1(x))
        output = self.final_act(self.deconv2(x))
        return output

```

Task 2.4: Joint Auto-Encoder Network

Implement the auto-encoder network by combining the encoder and the decoder. In the `__init__` function, instantiate an encoder from Task 2 and a decoder from Task 3. In `forward`, pass the input through the encoder and the decoder: $\mathbf{Y} = \mathcal{D}(\mathcal{E}(\mathbf{X}))$

```

class AutoEncoder(torch.nn.Module):
    """Full auto-encoder combining an encoder and a decoder module.

    Constructor args:
        Q1, Q2: the number of convolutional channels for layer 1 and 2 of the encoder network
        K: The dimensionality of the embedding vector
    """
    def __init__(self, Q1, Q2, K):
        super(AutoEncoder, self).__init__()
        # instantiate the two sub-networks
        self.encoder = Encoder(Q1, Q2, K)
        self.decoder = Decoder(Q1, Q2, K)

    def forward(self, x):
        """Run the input through the encoder and then the decoder.

        Args:
            x: An input batch of size Bx1xHxW where B is the batch size and H=W=28 is the input image dimension

        Returns:
            A batch of reconstructed images of size Bx1xHxW
        """

```

```
# encode input
deep_feature = self.encoder(x)
# decode to output
reconstructed = self.decoder(deep_feature)
return reconstructed
```

✓ Test 1: Output Sizes

The code below instantiates the auto-encoder network with $Q_1 = Q_2 = 32$ and $K = 10$. Then the given input $\mathbf{X}$ is provided to the (untrained) auto-encoder network. Use these codes to verify that the deep feature extracted by the encoder and the output from the decoder part both have the desired size. Also, verify that the output values are between 0 and 1.

If the tests cannot be passed, please check the implementations above.

```
# create network
model_ = AutoEncoder(4, 8, 3).to(device)

# create or select a sample
x = torch.randn((1,1,28,28), device=device)

# use encoder to encode image and check its size
deep_features_ = model_.encoder(x)
assert deep_features_.shape[1] == 3

# use decoder to generate an image and check its size and value range
output_ = model_.decoder(deep_features_)
assert output_.shape[2:] == (28,28)
assert torch.all(output_ >= 0) and torch.all(output_ <= 1)

# make sure that the auto-encoder is implemented correctly by comparing the
# end-to-end forward pass against running encoder then decoder separately
output__ = model_(x)
assert output__.shape[2:] == (28,28)
assert torch.allclose(output_, output__)
```

✓ Training and Evaluation

We will implement a training procedure for an auto-encoder network.

To train the network, we will use the L_2 distance between the output and the input of the network as a loss function, which is implemented in `torch.nn.MSELoss`:

$$\mathcal{J}^{L_2}(\mathbf{X}, \mathbf{Y}) = \|\mathbf{X} - \mathbf{Y}\|^2$$

For optimization, we will make use of the `Adam` optimizer with a learning rate of $\eta = 0.001$. We will run the training for 10 epochs and compute training and validation set loss after each epoch.

For evaluation, we will check whether some of the validation set samples are correctly reconstructed from the auto-encoder network by visualizing them.

In our test sets, there are 10000 in-distribution samples and 20800+10000 OOD samples. In the case of evaluating the success rate of anomaly detection, accuracy is not the best metric because it gives equal weights to two classes with unequal numbers of samples, while the actual distribution of the two classes is unbalanced. A system can achieve high accuracy by simply predicting the majority class for every instance, while completely ignoring the minority class.

In such cases, we choose to use the true positive rate (TPR) and true negative rate (TNR), which measure the proportion of actual positive (i.e., the known) or negative (i.e., the unknown) cases that are correctly identified by the system.

$$\text{TPR} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

$$\text{TNR} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$$

✓ Task 2.5 Training Loop

Instantiate the auto-encoder network with $Q_1 = 16$, $Q_2 = 32$ and $K = 10$.

To train the auto-encoder network, we will use the L_2 distance between the output and the input of the network as a loss function.

This loss function is implemented in `torch.nn.MSELoss` and make sure to use `reduction='none'` as this will allow us to compute the loss for each sample separately, which will become important later. This will keep the output size identical to the input size.

However, because we still need to compute the average loss for the backpropagation and to report the training and validation

performance, we need to manually compute these averages. For `sample_J`, compute the mean loss for each sample over the data dimensions. For `J` compute the average of `sample_J` over the batch size.

Since training an auto-encoder is tricky, we will make use of the Adam optimizer. Choose a learning rate of $\eta = 0.001$. Implement the training loop for 10 epochs.

Note: If the training and validation loss does not decrease during training, try to reduce the learning rate (to $\eta = 0.0005$ or even lower) and re-start the training (remember to re-initialize the network, too).

```
# Initialize model
model = AutoEncoder(Q1=16, Q2=32, K=10).to(device)

# Number of epochs
num_epochs = 10

# Loss function and optimizer
loss_function = torch.nn.MSELoss(reduction='none')
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# progress bar for the training/validation iterations
from tqdm import tqdm

# Training loop
def train():
    """Run one epoch of training and return the per-sample losses.

    Returns:
    The list of loss values per training sample
    """
    # put the model into training mode (enables dropout/batch-norm training behavior)
    model.train()
    losses = []
    # iterate over mini-batches from the training loader
    for X, _ in tqdm(train_loader):
        X = X.to(device)

        # Forward pass
        Y = model(X)
        # Compute loss per sample. sample_J is a tensor of shape B
        sample_J = loss_function(Y, X).mean(dim=[1, 2, 3])
        # Compute average loss over batch
        J = sample_J.mean()

        # Backward and optimize
        optimizer.zero_grad()
        J.backward()
        optimizer.step()

        # Add sample-wise loss to list
        losses.append(sample_J.detach().cpu())

    # concatenate all per-batch tensors into a single 1D tensor
    losses = torch.cat(losses)
    return losses

# test (i.e. Inference)
def test(data_loader):
    """Evaluate the model on a given data loader and return the per-sample losses.

    Args:
    data_loader: The test dataloader for which to evaluate the model

    Returns:
    The list of loss values per test sample
    """
    # put the model into evaluation mode
    model.eval()
    losses = []
    # disable gradient computation since we are not updating weights here
    with torch.no_grad():
        for X, _ in tqdm(data_loader):
            X = X.to(device)

            # Forward pass
            Y = model(X)

            # Compute loss per sample. sample_J is a tensor of shape (batch_size,)
            sample_J = loss_function(Y, X).mean(dim=[1, 2, 3])

            # Add sample-wise loss to list
```

```

        losses.append(sample_j.cpu())

    losses = torch.cat(losses)
    return losses

```

✓ Test 2: Loss Computation

We test that the shape of the loss values is correct to assure that the reduction is only applied to each image and the loss for each sample is reported separately.

With the default parameters from the tasks above, you can expect a loss of roughly 0.015-0.020 for the training and in-distribution test set after 10 epochs.

```

class RandomImageDataset(torch.utils.data.Dataset):
    """Synthetic dataset of random images, used only for verifying output shapes."""
    def __init__(self, shape):
        self.shape = shape
        # Generate random samples and targets
        self.data = torch.rand(*self.shape) # Random values between 0 and 1
        self.targets = torch.randint(0, 10, (self.shape[0],)) # Random integer labels from 0 to 9

    def __len__(self):
        # total number of samples in the dataset
        return self.shape[0]

    def __getitem__(self, idx):
        # guard against out-of-range access
        if idx >= self.shape[0]:
            raise IndexError(f"Index {idx} out of range for dataset of size {self.shape[0]}")
        return self.data[idx], self.targets[idx]

# build a tiny dataset with a handful of random images
data_size_ = 5
dataset_ = RandomImageDataset(shape=(data_size_, 1, 28, 28))
dataloader_ = DataLoader(dataset_, batch_size=data_size_, shuffle=False)
# run inference on the random images to check the loss shape
losses_ = test(dataloader_)

# the loss vector should contain exactly one entry per sample
assert losses_.shape == (data_size_,)

```

```

100%|██████████| 1/1 [00:00<00:00, 6.87it/s]

```

Now, we train the model on our training dataset, and perform the validation on our validation dataset. Since this is the same as in previous exercises, we have implemented this boring part for you.

```

# Training and validation
# accumulate mean loss per epoch for later plotting
train_losses = []
validation_losses = []

print("Starting training...")
# iterate over the configured number of epochs
for epoch in range(num_epochs):
    # run one pass through the training data and take the average loss
    train_loss = train().mean()
    # evaluate on the test set to track generalization
    validation_loss = test(test_loader).mean()

    train_losses.append(train_loss)
    validation_losses.append(validation_loss)

# report the progress after each epoch
print(f'Epoch [{epoch+1}/{num_epochs}], Average Train Loss: {train_loss:.6f}, Average Validation Loss: {validation_loss:.6f}')

```

```

Starting training...
100%|██████████| 469/469 [00:12<00:00, 36.50it/s]
100%|██████████| 79/79 [00:01<00:00, 47.02it/s]
Epoch [1/10], Average Train Loss: 0.052058, Average Validation Loss: 0.027999
100%|██████████| 469/469 [00:08<00:00, 53.52it/s]
100%|██████████| 79/79 [00:01<00:00, 49.96it/s]
Epoch [2/10], Average Train Loss: 0.024671, Average Validation Loss: 0.022721
100%|██████████| 469/469 [00:09<00:00, 51.22it/s]
100%|██████████| 79/79 [00:01<00:00, 64.83it/s]
Epoch [3/10], Average Train Loss: 0.021096, Average Validation Loss: 0.019962
100%|██████████| 469/469 [00:09<00:00, 49.30it/s]
100%|██████████| 79/79 [00:01<00:00, 62.70it/s]
Epoch [4/10], Average Train Loss: 0.019050, Average Validation Loss: 0.018620
100%|██████████| 469/469 [00:09<00:00, 49.33it/s]

```

```

100%|██████████| 79/79 [00:01<00:00, 63.62it/s]
Epoch [5/10], Average Train Loss: 0.017777, Average Validation Loss: 0.017440
100%|██████████| 469/469 [00:09<00:00, 48.99it/s]
100%|██████████| 79/79 [00:01<00:00, 63.50it/s]
Epoch [6/10], Average Train Loss: 0.016861, Average Validation Loss: 0.016786
100%|██████████| 469/469 [00:09<00:00, 48.65it/s]
100%|██████████| 79/79 [00:01<00:00, 62.53it/s]
Epoch [7/10], Average Train Loss: 0.016214, Average Validation Loss: 0.016158
100%|██████████| 469/469 [00:09<00:00, 49.53it/s]
100%|██████████| 79/79 [00:01<00:00, 64.16it/s]
Epoch [8/10], Average Train Loss: 0.015684, Average Validation Loss: 0.015675
100%|██████████| 469/469 [00:09<00:00, 51.46it/s]
100%|██████████| 79/79 [00:01<00:00, 47.00it/s]
Epoch [9/10], Average Train Loss: 0.015281, Average Validation Loss: 0.015406
100%|██████████| 469/469 [00:10<00:00, 46.48it/s]
100%|██████████| 79/79 [00:01<00:00, 45.15it/s]Epoch [10/10], Average Train Loss: 0.014939, Average Validation Loss: 0.01507

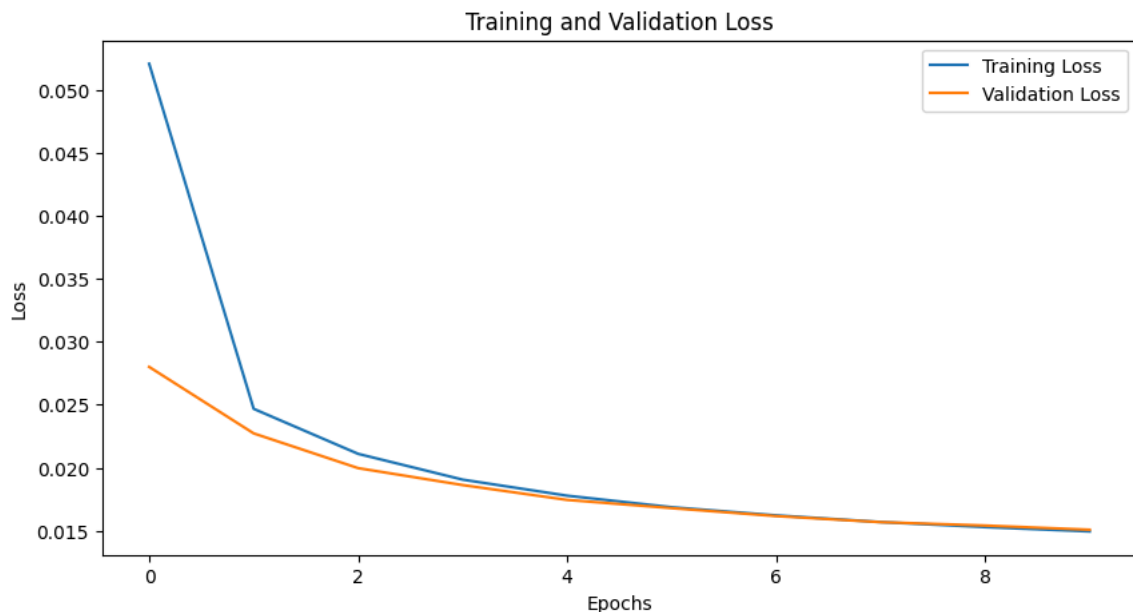
```

Additionally, we plot the training and validation set losses.

```

# Plot training and validation loss
pyplot.figure(figsize=(10, 5))
pyplot.plot(train_losses, label='Training Loss')
pyplot.plot(validation_losses, label='Validation Loss')
pyplot.xlabel('Epochs')
pyplot.ylabel('Loss')
pyplot.title('Training and Validation Loss')
pyplot.legend()
pyplot.show()

```



✖ Out-of-Distribution (OOD) Detection

In this task, perform out-of-distribution (OOD) detection via the reconstruction error from the trained network. We will use the test set of EMNIST letters and FashionMNIST as OOD samples. We will first reconstruct the images of the three test sets. Then we will perform OOD detection by computing the sample-wise loss thresholding the reconstruction error. Samples with an error below a certain threshold are considered in-distribution, while samples with an error above the threshold are considered OOD.

Forward the re-assigned loss as predictions into the function defined above to compute TPR and TNR. Compute the regular accuracy as well to make a comparison.

✖ Task 2.6: Reconstruction Result

This task is to visualize the reconstructed images from their originals. For this purpose, load the first batch of each of the test sets. Forward the images through the trained auto-encoder network to extract their reconstructions. Make a single plot with 2 rows and 10 columns. In the 1st row, plot the original samples and in the 2nd row plot the corresponding reconstructed samples.

Then, for each dataset, compute the average reconstruction error of the entire dataset and print it. Hint: you can reuse the `test()` function from task 2.5.

You should be able to see that the reconstruction error is much larger for the OOD samples (especially for far-OOD samples) than for the in-distribution samples. This is a good sign that the auto-encoder network is able to detect OOD samples.

```
# Visualization of some reconstructed images
def visualize_reconstruction(data_loader, data_set_name, num_images=10):
    """Display a row of original images alongside their reconstructions.

    Args:
    data_loader: The data loader to sample images to be reconstructed
    data_set_name: The name of the dataset, for displaying purposes only
    num_images: The number of images to reconstruct and display
    """
    assert num_images <= batch_size, "num_images should be less than or equal to batch size"
    # put the model into evaluation mode
    model.eval()
    with torch.no_grad():
        # Get a batch of test images
        X, _ = next(iter(data_loader))
        X = X.to(device)

        # Generate reconstructions
        Y = model(X)

        # Move tensors to CPU
        X = X.cpu()
        Y = Y.cpu()

        # Plot original vs reconstructed images
        fig, axs = pyplot.subplots(2, num_images, figsize=(num_images, 3))
        fig.suptitle(f"Reconstruction on {data_set_name}")
        # flatten the 2D grid of axes so we can index it linearly
        axs = axs.flatten()

        for i in range(num_images):
            # Original images in the top row
            ax = axs[i]
            ax.imshow(X[i][0], cmap='gray')
            ax.set_title("Original", fontsize=8)
            ax.axis('off')

            # Reconstructed images in the bottom row
            ax = axs[i + num_images]
            ax.imshow(Y[i][0], cmap='gray')
            ax.set_title("Reconstructed", fontsize=8)
            ax.axis('off')

        pyplot.tight_layout()
        pyplot.show()

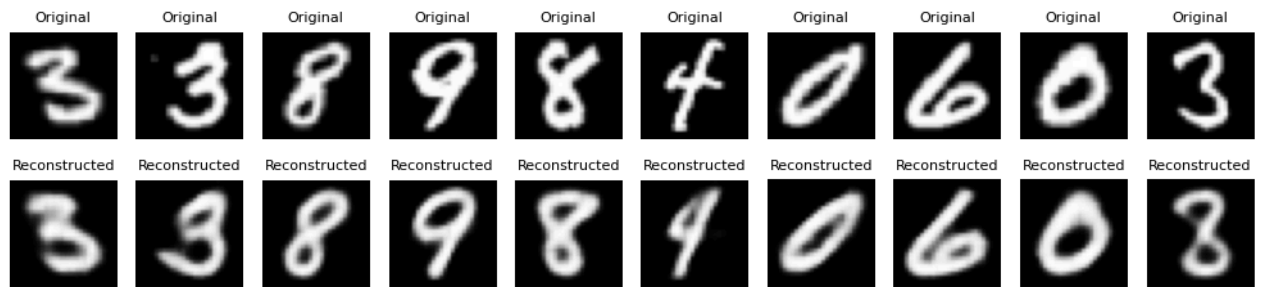
# visualize reconstructions and compute the average reconstruction error on the in-distribution test set
visualize_reconstruction(test_loader, "Test Set")
reconstruction_error_test = test(test_loader).mean()
print(f'Average Test Reconstruction Error: {reconstruction_error_test:.6f}')

# Visualize some open-set samples for EMNIST letters and FashionMNIST

# EMNIST letters
visualize_reconstruction(emnist_letters_loader, "EMNIST Letters")
# Evaluate reconstruction error on open-set samples
reconstruction_error_emnist = test(emnist_letters_loader).mean()
print(f'Average Open-set Test Reconstruction Error on EMNIST Letters: {reconstruction_error_emnist:.6f}')

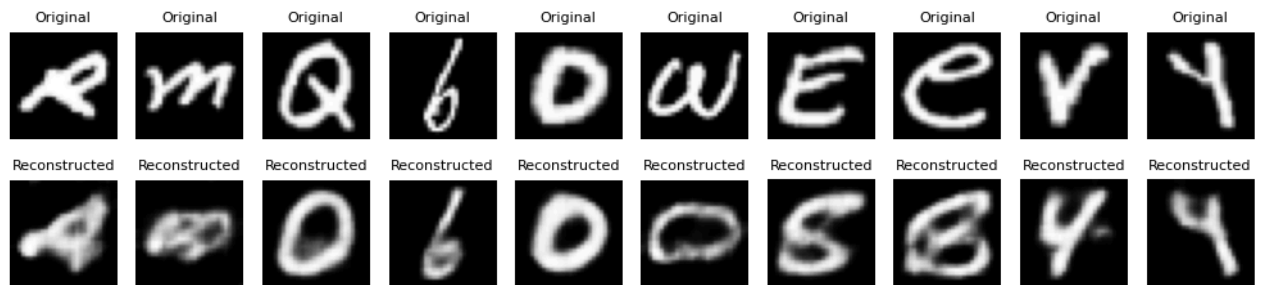
# FashionMNIST
visualize_reconstruction(fashion_mnist_loader, "FashionMNIST")
# Evaluate reconstruction error on open-set samples
reconstruction_error_fashion = test(fashion_mnist_loader).mean()
print(f'Average Open-set Test Reconstruction Error on Fashion MNIST: {reconstruction_error_fashion:.6f}')
```

Reconstruction on Test Set



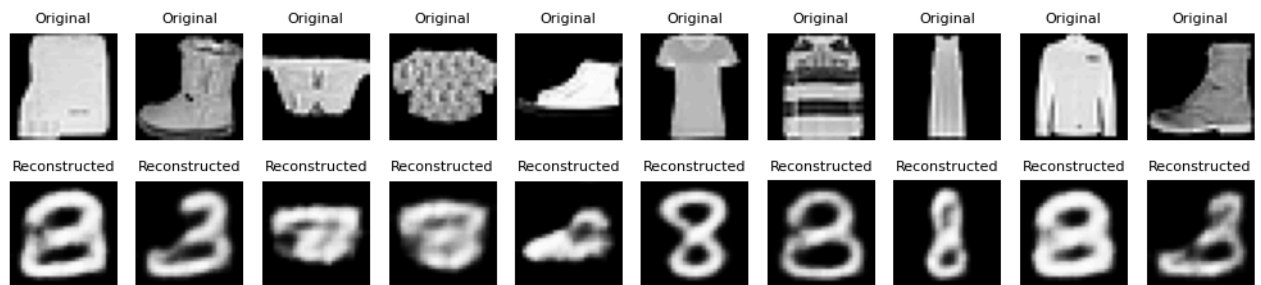
100%|██████████| 79/79 [00:01<00:00, 63.23it/s]
Average Test Reconstruction Error: 0.015075

Reconstruction on EMNIST Letters



100%|██████████| 163/163 [00:02<00:00, 60.22it/s]
Average Open-set Test Reconstruction Error on EMNIST Letters: 0.039089

Reconstruction on FashionMNIST



100%|██████████| 79/79 [00:01<00:00, 62.75it/s] Average Open-set Test Reconstruction Error on Fashion MNIST: 0.075166

Task 2.7: Visualizing the OOD Signal

We now exploit the inability of the AE network to reconstruct OOD samples, by using the reconstruction error as a signal for OOD detection and applying a threshold to the reconstruction error. Determine a reasonable `ood_threshold` from the plot of this task (you need to run the plotting with value 0 first, and then you can decide for a good threshold) that can be used to classify the samples as in-distribution or OOD. Compute and plot the distribution of test errors on all three datasets as a histogram along with the threshold, in order to visualize the OOD detection performance.

```
# Plot a histogram overlay of reconstruction errors for the three datasets, with a threshold line.

# define a reasonable threshold for distinguishing ID and OOD samples, using this plot
# make sure to set this threshold to a reasonable value before continuing to the next cell
ood_threshold = 0.018

# gather per-sample reconstruction errors for each dataset
test_errors = test(test_loader)
emnist_letters_errors = test(emnist_letters_loader)
fashion_mnist_errors = test(fashion_mnist_loader)

pyplot.figure(figsize=(10, 5))
# combine all errors to determine a common range for the histogram bins
all_errors = torch.cat([torch.as_tensor(test_errors), torch.as_tensor(emnist_letters_errors), torch.as_tensor(fashion_mnist_errors)])
bins = torch.linspace(0, all_errors.max(), 50)

# overlay the three histograms with partial transparency
pyplot.hist(test_errors, bins=bins, alpha=0.5, color='green', label='MNIST')
pyplot.hist(emnist_letters_errors, bins=bins, alpha=0.5, color='blue', label='Near-OOD Samples')
pyplot.hist(fashion_mnist_errors, bins=bins, alpha=0.5, color='red', label='Far-OOD Samples')
```

```
# mark the threshold as a vertical dashed line (not shown in the example plot)
if ood_threshold is not None:
    pyplot.axvline(ood_threshold, color='black', linestyle='--', label='Threshold')
pyplot.xlabel('Reconstruction Error')
pyplot.ylabel('Frequency')
pyplot.title('Test Reconstruction Error Distributions')
pyplot.legend()
pyplot.show()
```

```
100%|██████████| 79/79 [00:01<00:00, 60.44it/s]
100%|██████████| 163/163 [00:03<00:00, 46.68it/s]
100%|██████████| 79/79 [00:01<00:00, 64.64it/s]
```



✓ Task 2.8: True Positive/Negative Rate Calculation

Define a function that takes the predictions and truth values, as lists, and returns TPR and TNR.

You can use `sklearn.metrics.confusion_matrix` (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html) to compute true positives, true negatives, false positives, and false negatives, or compute them by their definitions.

```
from sklearn.metrics import confusion_matrix

def compute_tpr_tnr(predictions, labels):
    """Compute the true positive rate and true negative rate from predictions and ground-truth labels.

    Args:
        predictions: The predicted classes (-1 or 1) in shape B
        labels: The ground-truth labels (-1 or 1) in shape B

    Returns:
        True Positive Rate
        True Negative Rate
    """
    # Compute the confusion matrix, directly from the torch tensors
    predictions_np = predictions.cpu().numpy()
    labels_np = labels.cpu().numpy()

    cm = confusion_matrix(labels_np, predictions_np, labels=[-1, 1])

    tn, fp, fn, tp = cm[0,0], cm[0,1], cm[1,0], cm[1,1]

    # Compute TPR and TNR
    tpr = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    tnr = tn / (tn + fp) if (tn + fp) > 0 else 0.0

    return tpr, tnr
```

✓ Test 3: TPR & TNR Calculation Check

With the given labels and predictions, call the function defined in Task 2.8 and check the returned TPR and TNR are correct.

```

labels_ = torch.tensor([1, 1, 1, 1, 1, 1, -1, 1, 1, -1, -1, 1, 1, -1, 1, -1, 1, 1, 1])
predictions_ = torch.tensor([1, -1, 1, 1, 1, 1, -1, 1, 1, -1, 1, 1, 1, -1, 1, -1, 1, 1, 1])

# Compute TPR and TNR
tpr_, tnr_ = compute_tpr_tnr(predictions_, labels_)

assert(abs(tpr_ - 14/15) < 1e-8)
assert(abs(tnr_ - 0.8) < 1e-8)

```

Task 2.9: Evaluating OOD Detection

Now we will perform OOD detection by computing the sample-wise loss. In order to evaluate the performance, we need to convert the labels into binary labels and assign binary predictions. Assign 1 to samples with a reconstruction error smaller than your `ood_threshold`, and -1 otherwise. We adjust the labels accordingly. Then, compute the TPR and TNR for the three datasets.

```

predictions = []
labels = []
for dataloader in [test_loader, emnist_letters_loader, fashion_mnist_loader]:
    losses = test(dataloader)

    binary_prediction = torch.where(losses <= ood_threshold, torch.tensor(1), torch.tensor(-1))

    if dataloader == test_loader:
        binary_labels = torch.full_like(losses, 1)
    else:
        binary_labels = torch.full_like(losses, -1)

    predictions.append(binary_prediction)
    labels.append(binary_labels)

predictions = torch.cat(predictions)
truth = torch.cat(labels)

tpr, tnr = compute_tpr_tnr(predictions, truth)
print(f'True Positive Rate (TPR): {tpr:.6f}')
print(f'True Negative Rate (TNR): {tnr:.6f}')

```

```

100%|██████████| 79/79 [00:01<00:00, 61.85it/s]
100%|██████████| 163/163 [00:02<00:00, 62.49it/s]
100%|██████████| 79/79 [00:01<00:00, 68.42it/s]
True Positive Rate (TPR): 0.702200
True Negative Rate (TNR): 0.862403

```

Open-Set Recognition

So far we have ignored the class labels of the training data and only made the binary distinction between in and out-of-distribution samples. In this task, we will use the class labels of the training data to perform open-set recognition, which also considers the class labels of the in-distribution samples. We will show that we can solve the in-distribution classification task with the encoder network that was trained in an unsupervised fashion.

Task 2.10: OSR via Class Distance

We will perform the in-distribution classification task by computing class centroids, i.e. the average deep feature representations of the training data for each class. Then we will compute the Euclidean distance between the deep feature representation of the test sample and the class centroids. The class with the smallest distance is assigned to the test sample. Instead of using the reconstruction error as a signal of OODness, we will use the distance between the deep feature representation of the sample and the class centroids as signal of OODness and apply a threshold to the minimal distance of any test sample to all class centroids.

- Compute the class centroids for the training set.
 - Store them in a torch tensor of shape (O, K) , where K is the deep feature dimension and O is the number of known classes.
 - It is advised to compute a dictionary first with the class labels as keys and a list of all the deep feature representations as values, then compute the mean of each list and store it in the tensor.
- Compute the distances of each sample to each class center. Use the `torch.cdist` function to compute the Euclidean distances for an entire batch at once.
- Predict the classes for the test samples as follows:
 - If for a sample the minimum distance to any class centroid is larger than the `osr_threshold`, assign it the label `-1` (unknown).
 - Otherwise, assign it the label of the class centroid with the smallest distance.

```

class OSRClassifier():
    """Open-set recognition classifier based on distances to class centroids in deep feature space."""
    def __init__(self, autoencoder):
        # keep references to the encoder and decoder of the trained auto-encoder
        self.encoder = autoencoder.encoder
        self.decoder = autoencoder.decoder

    def compute_centroids(self):
        """Compute and store the centroid of deep features for each class."""
        # put the encoder into evaluation mode
        self.encoder.eval()
        # mapping from class label to a collection of deep features
        class_features = {i: [] for i in range(10)}
        with torch.no_grad():
            for x, t in train_loader:
                x = x.to(device)
                # encode the batch into deep features
                deep_features = self.encoder(x)
                for i, label in enumerate(t):
                    # append features for this label
                    class_features[label.item()].append(deep_features[i].cpu())

        # Initialize centroids in reasonable data structure
        self.centroids = torch.zeros(10, self.encoder.fc.out_features) # 10 classes, K dimensions
        # Compute the mean of the deep features for each class
        for label in class_features:
            if class_features[label]:
                self.centroids[label] = torch.stack(class_features[label]).mean(dim=0)
        self.centroids = self.centroids.to(device)

    def compute_distances(self, data_loader):
        """Compute the distances from each sample's deep feature to all stored centroids.

        Args:
            data_loader: The test set data loader to use

        Returns:
            distances: The distances of all samples to all class means, in shape Nx0
            labels: The correct class label, in shape N
        """
        self.encoder.eval()
        distances = []
        labels = []
        with torch.no_grad():
            for X, T in data_loader:
                X = X.to(device)
                # extract deep features from the autoencoder
                deep_features = self.encoder(X)

                # Compute distances to all centroids for each sample
                dist = torch.cdist(deep_features, self.centroids)
                distances.append(dist.cpu())
                labels.append(T.cpu())

        # stack the per-batch results into single tensors
        distances = torch.cat(distances)
        labels = torch.cat(labels)

        return distances, labels

    def predict(self, data_loader, threshold):
        """Predict class labels, assigning -1 when the sample is considered out-of-distribution.

        Args:
            data_loader: The test set data loader to use
            threshold: The threshold to be used for unknown class

        Returns:
            predictions: The predicted class, including the unknown class -1, in shape N
            labels: The correct class label, in shape N
        """
        self.encoder.eval()
        # compute distances to the stored centroids
        distances, labels = self.compute_distances(data_loader)

        min_distances, predicted_classes = torch.min(distances, dim=1)

        # Make predictions based on the threshold
        predictions = torch.where(min_distances > threshold, torch.tensor(-1), predicted_classes)

        return predictions, labels

```

```

def plot_centroid_reconstruction(self, num_images=10):
    """Decode each class centroid and display the resulting images."""
    # We have implemented this for you.
    self.encoder.eval()
    with torch.no_grad():

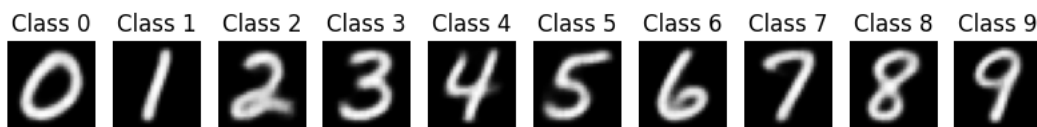
        # pass the centroids through the decoder to obtain images
        reconstruction = self.decoder(self.centroids.clone().to(device)).cpu()

        # Plot the centroids
        fig, axs = pyplot.subplots(1, num_images, figsize=(num_images, 2))
        fig.suptitle("Centroid Reconstructions")
        for i in range(num_images):
            axs[i].imshow(reconstruction[i][0], cmap='gray')
            axs[i].set_title(f"Class {i}")
            axs[i].axis('off')
        pyplot.show()

    # Create OSR classifier for the autoencoder
    osr_classifier = OSRClassifier(model)
    # compute centroids for the training set
    osr_classifier.compute_centroids()
    # Plot reconstructed centroids
    osr_classifier.plot_centroid_reconstruction(num_images=10)

```

Centroid Reconstructions



Task 2.11: Visualization of Distances to Class Centroids

Plot the distances of the test samples to the class centroids as a histogram. Make sure to plot the distances for the three datasets in a single plot. Assign `ood_threshold=None` in the first round to plot the distributions. Then, set it to a reasonable value and call this plot again.

```

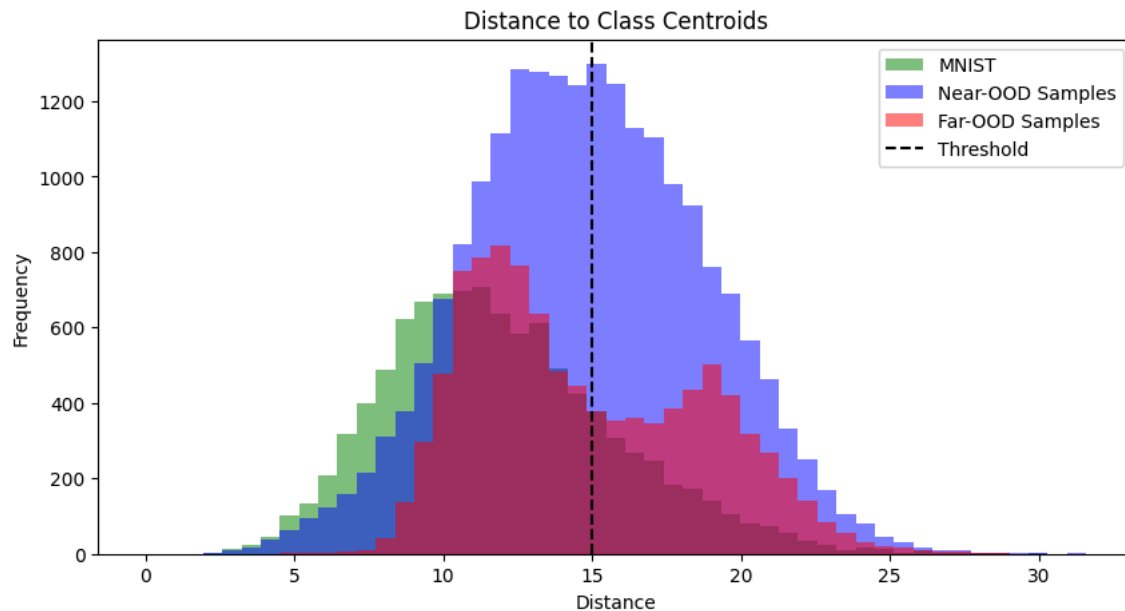
# Set this threshold to a reasonable value before going on with the next cell
osr_threshold = 15

# gather the distance tensors for each dataset
test_distances, _ = osr_classifier.compute_distances(test_loader)
emnist_letters_distances, _ = osr_classifier.compute_distances(emnist_letters_loader)
fashion_mnist_distances, _ = osr_classifier.compute_distances(fashion_mnist_loader)

# compute min distances per sample over all classes
min_test_distances = torch.min(test_distances, dim=1).values
min_emnist_letters_distances = torch.min(emnist_letters_distances, dim=1).values
min_fashion_mnist_distances = torch.min(fashion_mnist_distances, dim=1).values

pyplot.figure(figsize=(10, 5))
# concatenate distances to determine a common histogram range
all_distances = torch.cat([min_test_distances, min_emnist_letters_distances, min_fashion_mnist_distances])
bins = torch.linspace(0, all_distances.max(), 50)
# overlay histograms with partial transparency for comparison
pyplot.hist(min_test_distances, bins=bins, alpha=0.5, color='green', label='MNIST')
pyplot.hist(min_emnist_letters_distances, bins=bins, alpha=0.5, color='blue', label='Near-OOD Samples')
pyplot.hist(min_fashion_mnist_distances, bins=bins, alpha=0.5, color='red', label='Far-OOD Samples')
# draw the threshold as a vertical dashed line (not shown in the example plot)
if osr_threshold is not None:
    pyplot.axvline(osr_threshold, color='black', linestyle='--', label='Threshold')
pyplot.xlabel('Distance')
pyplot.ylabel('Frequency')
pyplot.title('Distance to Class Centroids')
pyplot.legend()
pyplot.show()

```



Task 2.12: OSR Evaluation

In this task, we will evaluate the performance of the open-set recognition system on the test sets. We will use the same evaluation metrics as in the OOD detection task, i.e. TPR and TNR. In addition, we compute the closed-set accuracy on the known classes and plot the corresponding confusion matrix. Make sure to only compute the accuracy and confusion matrix on the known classes, i.e. the samples with labels 0-9.

Note: There exist evaluation metrics such as the OSCR (Open Set Classification Rate) that are more suitable for open-set recognition tasks, but we will use the TPR and TNR metrics for simplicity here.

```
# compute TPR and TNR on all datasets
predictions = []
labels = []
# iterate over each dataset and accumulate the predictions and labels
for dataloader in [test_loader, emnist_letters_loader, fashion_mnist_loader]:
    pred, label = osr_classifier.predict(dataloader, osr_threshold)
    predictions.append(pred)
    labels.append(label)

# move the concatenated tensors to CPU for use with sklearn/matplotlib
predictions = torch.cat(predictions).cpu()
labels = torch.cat(labels).cpu()

# Compute binary predictions and labels: 1 for in-distribution, -1 for out-of-distribution
binary_labels = torch.where(labels == -1, torch.tensor(-1), torch.tensor(1))
binary_predictions = torch.where(predictions == -1, torch.tensor(-1), torch.tensor(1))

# Compute TPR and TNR
tpr, tnr = compute_tpr_tnr(binary_predictions, binary_labels)
print(f'True Positive Rate (TPR): {tpr:.6f}')
print(f'True Negative Rate (TNR): {tnr:.6f}')

# plot confusion matrix, including unknown class
figure = pyplot.figure(figsize=(10, 8))
ConfusionMatrixDisplay.from_predictions(
    labels,
    predictions,
    labels=list(range(-1,10)), # -1 for OOD, then 0-9 for classes
    normalize='true',
    ax=figure.gca(),
    values_format=".2f"
)
pyplot.savefig("Assignment04_Task2.12_confusion_matrix.pdf", bbox_inches="tight")
pyplot.show()
```

True Positive Rate (TPR): 0.797500
True Negative Rate (TNR): 0.457597

